BANKING LOAN RISK ANALYSIS APPLICATION

Design Document | Python Full-Stack and Data Engineering Capstone

Submission deliverable: Design_Document.pdf | October 2026

Document field	Description
Project	Banking Loan Risk Analysis Application (credit-card repayment dataset)
Implementation	Python / Pandas ETL, MySQL, FastAPI, Streamlit / Plotly
Risk method	Deterministic, explainable business rules; no machine learning
Deployment	Locally runnable backend, frontend, and MySQL database
Scope	System architecture, data flow, technology decisions, security, testing, limitations

1. Problem Statement and Business Need

Banking teams need a repeatable method to clean historical credit records, analyze customer repayment behaviour, identify review-priority accounts, and report portfolio exposure. Manual spreadsheet workflows make reconciliation, auditability, role separation, and consistent risk segmentation difficult. This capstone implements an end-to-end data engineering and analytics system using the UCI Default of Credit Card Clients dataset. Although the project title uses loan risk, the underlying records are credit-card accounts; the application does not originate loans or approve credit.

2. Objectives and Scope

Objectives: validate and ingest source data; normalize customer, credit, and monthly activity records; perform full and incremental ETL with batch audit; compute explainable risk bands; provide authenticated customer CRUD and portfolio APIs; visualize customer and segment metrics; export filtered data; and verify the system with automated and end-to-end tests.

In scope: local historical batch processing, relational data storage, business-rule scoring, reporting, JWT-based access control, and CSV exports. **Out of scope:** AI/ML default prediction, RAG, embeddings, autonomous agents, real-time core banking integration, automated credit decisions, and production cloud deployment.

3. High-Level Architecture Design

The following architecture is the finalized system diagram. It illustrates the source-to-ETL-to-database-to-application flow, with ETL audit, deterministic scoring, authentication and authorization, and analytical user workflows. Arrows are logical relationships rather than an exact runtime call graph.

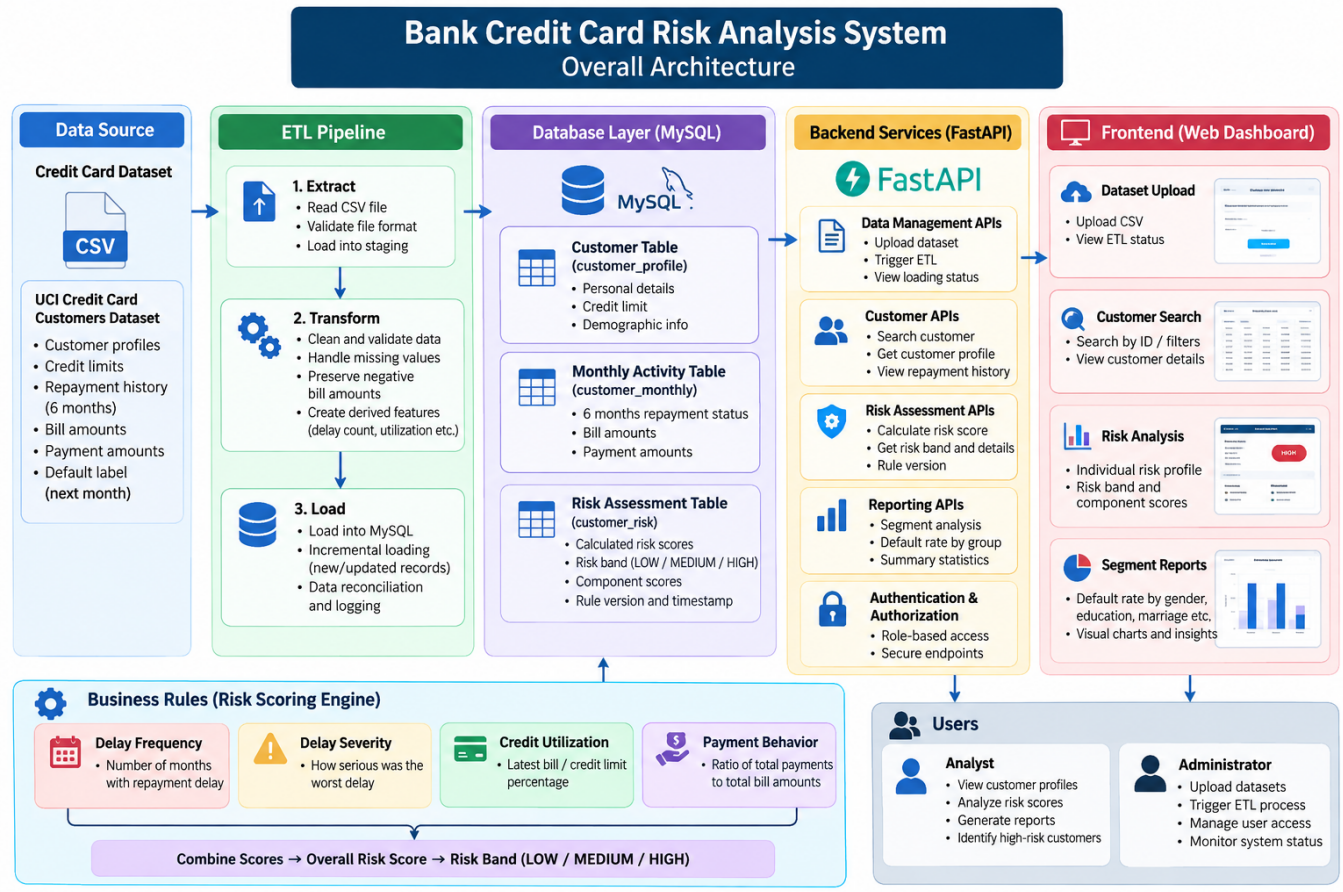


Figure 1. Banking Loan Risk Analysis system architecture (approved architecture artifact).

4. Architecture Explanation and Data Flow

Ingestion: the source Excel workbook is extracted and validated with Pandas, transformed into curated records, and loaded into MySQL. Full loading initializes the warehouse; incremental loading detects new, changed and unchanged customer records, applies transactional updates and records batch audit results.

Risk computation: the batch risk loader processes transformed source credit data, applies the versioned deterministic scoring rules and persists risk assessments in MySQL. The diagram places the risk engine alongside the warehouse to express its logical dependency on curated credit information; the implemented batch loader reads and transforms the source dataset rather than querying MySQL as its risk input.

Application: authenticated users operate Streamlit dashboards, which call FastAPI endpoints using Bearer JWTs. FastAPI enforces role permissions and queries MySQL for customer, credit, risk and analytics results. The UI does not directly access MySQL.

5. Technology Selection and Justification

Technology	Purpose and design rationale
Python + Pandas	Readable, reproducible extraction, validation, transformation and record-level comparisons for a 30,000-record source.
MySQL	Relational integrity, indexed joins, normalized dimensions and transactional batch updates.
FastAPI + Pydantic	Typed REST API contracts, validation, built-in OpenAPI specification and reusable backend services.
Streamlit + Plotly	Fast development of interactive portfolio and customer dashboards for local demonstration.
Argon2 + JWT	Password hashing and stateless authenticated API access with role checks.
Pytest + structured logs	Regression evidence, reconciliation checks and operational troubleshooting.

Applicability note: the generic submission template mentions LLMs, embeddings, vector databases, retrieval and agents. Those components are not part of the assigned non-AI Project 13 brief and are therefore intentionally not included. The equivalent solution design here is ETL, relational analytics, deterministic rules, API and dashboard.

6. Data Engineering Design

6.1 Source, validation and transformation

The UCI credit-card default dataset contains 30,000 customer records, with demographic codes, credit limit, six months of repayment status, six monthly bill amounts, six payment amounts, and a historical next-month default indicator. Data-quality checks cover missing and duplicate records or IDs, invalid limits, gender, age and repayment codes. Undocumented education and marital-status codes are preserved and labeled Others in reports rather than silently discarded.

Transformation derives delay-month count, maximum delay status, payment-delay flag, six-month bill/payment totals and averages, and latest credit utilization. The six monthly observations per customer are normalized into separate monthly activity rows.

6.2 Full load, incremental load and audit

The initial warehouse load produces 30,000 customer records, 30,000 credit profiles and 180,000 monthly activity rows for the original dataset. Incremental ETL compares incoming transformed records with stored state, classifies them as new/changed/unchanged, writes only needed changes, and logs batch source/loaded/failure counts and execution status. Transaction rollback avoids partially committed failed batches. It is a file-driven batch workflow, not a continuously scheduled streaming service.

7. Relational Data Model

Table	Role
customers	Customer identity, demographic references and age
dim_gender / dim_education / dim_marital_status	Normalized demographic labels
credit_profiles	Credit limits, derived behaviour features, historical default outcome
monthly_credit_activity	Six month-level repayment, bill and payment records per customer
customer_risk_assessment	Risk components, total score, band and rule version
etl_batch_audit	Batch status, timestamps and record reconciliation

Table	Role
app_users	Username, role and hashed password for application login

8. Deterministic Risk Assessment

A transparent scoring engine combines four business-rule components. The historical next-month default outcome is retained for descriptive comparison and is **not** used to compute the score. Risk bands are prioritization indicators, not calibrated probabilities of default.

Component	Point range	Basis
Delay frequency	0–3	Number of months with repayment delay
Delay severity	0–3	Maximum observed repayment-delay status
Credit utilization	0–3	Latest billed amount relative to credit limit
Payment behaviour	0–2	Payment-to-bill relationship
Total	**0–11**	**Sum of component scores**

Risk bands: LOW 0–2; MEDIUM 3–5; HIGH 6–11. The engine persists component scores, total score, risk band and rule version so that classifications are inspectable and reproducible.

9. Backend API and Data Access

FastAPI exposes health checking, login/current-user endpoints, customer listing and CRUD, customer activity/risk details, portfolio summary, risk distribution, monthly trends, demographic segments and filtered CSV export. Customer lists support page/page_size, filters and sorting. The separate API_Documentation.pdf records endpoint-level contracts and representative requests/responses.

Route family	Purpose
GET /health	API and MySQL availability
POST /auth/login; GET /auth/me	Login and authenticated identity
/customers and /customers/{customer_id}	Paginated search, details and admin-only create/update/delete
/customers/{customer_id}/activity and /risk	Monthly history and persisted risk scores
/analytics/summary; /risk-distribution; /monthly-trends; /segments	Portfolio and segment analytics
GET /export/customers	Filtered CSV export for authorized users

10. Security and Access Control

Application passwords are stored as Argon2 hashes. Successful login issues a signed HS256 JWT with a configured lifetime (30 minutes in the project settings). Protected routes validate Bearer tokens and enforce server-side role checks. ADMIN can read and manage customer records and export; ANALYST can read, analyze and export; VIEWER is read-only. Environment variables contain database credentials and JWT configuration, which must not be committed to GitHub. These controls do not constitute a full production security audit.

11. Frontend and User Workflows

The Streamlit interface has six functional views: Overview, Risk Analysis, Portfolio Analytics, Customer Explorer, Customer Management, and Reports & Export. Plotly provides interactive charts; authenticated HTTP calls retrieve portfolio and customer data from FastAPI. Available actions depend on the user role, while backend authorization remains the source of enforcement.

12. Evaluation Strategy and Test Evidence

The project uses functional and regression tests rather than GenAI evaluation frameworks. Testing covers ETL and transformation rules, data quality, deterministic risk thresholds, API schemas and authorization, frontend behaviour, database reconciliation, CRUD workflows, and exports. The completed local regression run reported **672 tests passed**, **four subtests passed** and **one PytestCollectionWarning**; frontend statement coverage was reported as **83%**. These figures describe the tested local project state, not independent production certification.

Verification also included MySQL connectivity, authenticated role checks, source-to-warehouse count reconciliation, customer mutation flows and reporting endpoints. Structured logging and ETL audit rows help trace failures and incremental processing outcomes.

13. Challenges, Decisions and Trade-offs

Design decision	Advantage	Limitation
Rule-based risk instead of ML	Explainable, auditable, no model training	Heuristic thresholds; no probability calibration
MySQL normalized tables	Integrity and efficient relational analytics	Schema and database setup required
Streamlit frontend	Rapid interactive dashboard delivery	Less UI flexibility than a custom SPA
FastAPI boundary	Typed contracts and central RBAC	Separate service process to run
Incremental file-based ETL	Traceable changed-record updates	Not real-time; scheduling not demonstrated

14. Limitations and Future Improvements

The historical UCI dataset may not reflect current customer populations. Risk thresholds are descriptive business rules, not validated default-probability estimates. Risk-batch refresh and incremental ingestion are separate operational tasks and should be coordinated after data changes. Potential improvements include scheduled orchestration, production monitoring, migrations, performance benchmarking, additional reconciliation dashboards and more comprehensive deployment controls. No such future features are claimed as already implemented.

15. Submission Alignment and References

This document supplies the required problem statement, architecture diagram and explanation, technology rationale, evaluation/test strategy, and challenges/trade-offs. The standalone architecture PDF, source code, README, API documentation, dataset details and presentation are separate deliverables. GenAI-specific sections of the generic guidelines do not apply to this explicitly non-AI capstone.

References: Submission_Guidelines.pdf; Project 13 - Banking Loan Risk Analysis Application.pdf; UCI Machine Learning Repository, Default of Credit Card Clients (https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients); project implementation and recorded test results.